

LOG430

BrokerX - Documentation d'Architecture

Basé sur le modèle arc42

Élève :

Thevenet, Louis

Automne 2025

Table des matières

1 Avant-propos 3
2 Introduction et Objectifs 3
2.1 Panorama des exigences 3
2.2 Objectifs qualité 3
2.3 Parties prenantes (Stakeholders) 3
3 Contraintes d'architecture 3
4 Portée et contexte du système 4
4.1 Contexte métier 4
4.2 Contexte technique 6
5 Stratégie de solution 7
6 Domain-Driven Design (DDD) 7
6.1 Introduction au DDD 7
6.2 Bounded Contexts 7
6.3 Entités et Value Objects 7
6.4 Aggregates 8
6.5 Repositories 8
6.6 Domain Services 9
6.7 Ubiquitous Language 9
6.8 Architecture en couches (Layered Architecture) 9
7 Vue des blocs de construction 10
7.1 Vue d'ensemble du système (Niveau 1) 10
7.2 Architecture du domaine (Niveau 2) 11
7.3 Architecture événementielle pour le traitement des ordres 11
7.4 Vue d'exécution 12
7.5 Vue de déploiement 13
8 Concepts transversaux 14
8.1 Décisions d'Architecture 14
8.2 Exigences qualité 15
8.3 CI/CD (Intégration et Déploiement Continu) 15
9 Runbook 16
9.1 Vue d'ensemble des services 16
9.2 Déploiement 16
9.3 Surveillance et monitoring 17
9.4 Tests de charge 17
10 Guide d'utilisation 18
10.1 Frontend Web 18
10.2 API 23
11 Glossaire 25
12 Annexe 25
12.1 Décisions d'architecture 25

1 Avant-propos

Les figures sont trouvables au format SVG dans le répertoire `docs/arc42/`.

2 Introduction et Objectifs

2.1 Panorama des exigences

L'application « BrokerX » est dotée d'une interface web faisant appel à une passerelle API sécurisée faisant elle-même appel à différents micro-services. Elle permet aux particuliers et petites entreprises d'accéder à une plateforme de courtage moderne. Les clients peuvent ainsi :

- Gérer un portefeuille personnel (solde, historique),
- Passer des ordres d'achat/vente d'instruments financiers (actions),
- Consulter les données de marché en temps réel.

2.2 Objectifs qualité

Priorité	Objectif qualité	Scénario
1	**Disponibilité**	$99,9\%$
2	**Performance**	Latence P95 (ordre $\rightarrow$ ACK) ≤ 100 ms & Débit ≥ 1200 ordres/s
3	**Observabilité**	Logs + métriques (4 Golden Signals)

Tableau 1. – Objectifs qualité

2.3 Parties prenantes (Stakeholders)

- **Clients** : utilisateurs via interface web.
- **Opérations Back-Office** : gestion des règlements, supervision.
- **Risque** : surveillance pré- et post-trade.
- **Fournisseurs de données de marché** : cotations en temps réel et simulateur d'exécution.

3 Contraintes d'architecture

Contrainte	Description
Technologie	Les plateformes/langages serveur privilégiées : Java, C#, Go, Rust ou C++
Architecture	Architecture microservices avec API Gateway
Traitement des ordres	Saga chorégraphiée distribuée via message broker (Redis)

Tableau 2. – Contraintes d'architecture

4 Portée et contexte du système

4.1 Contexte métier

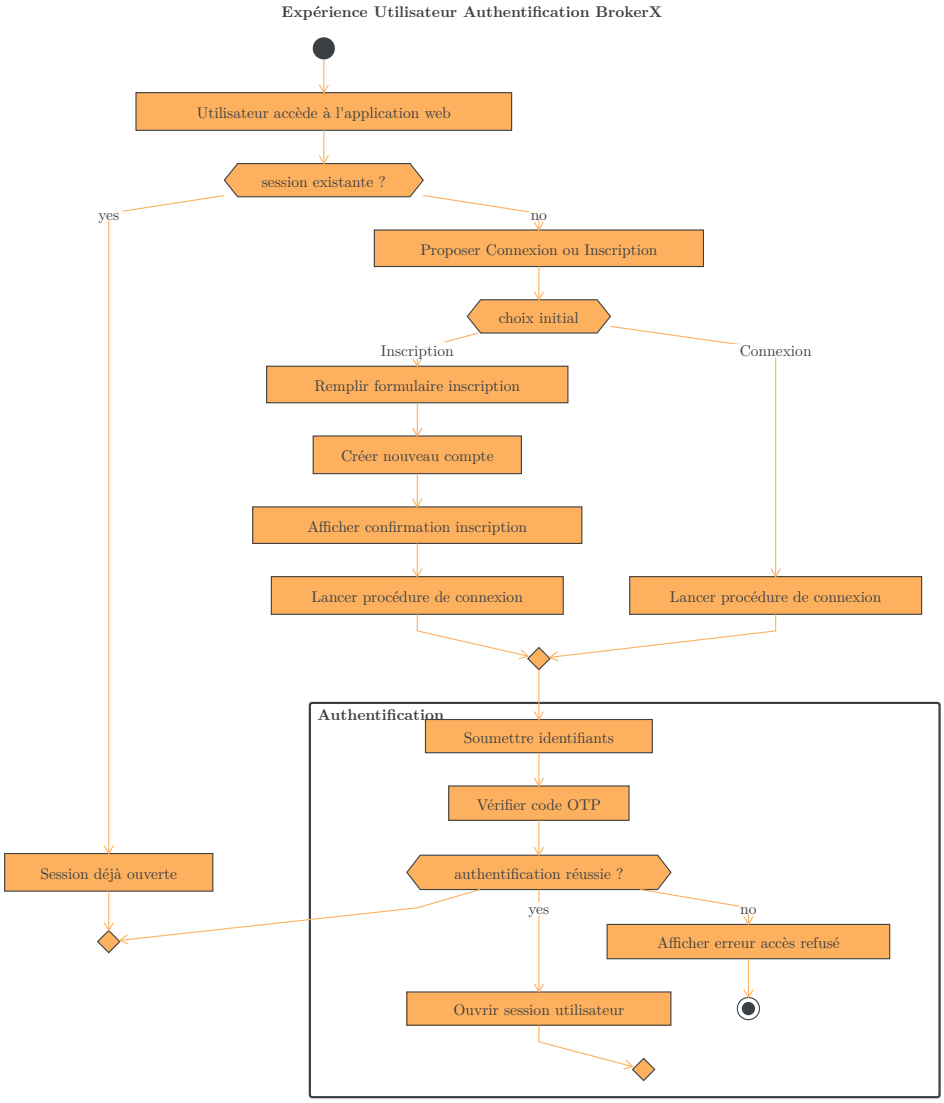


Fig. 1. – Authentication Activity Diagram

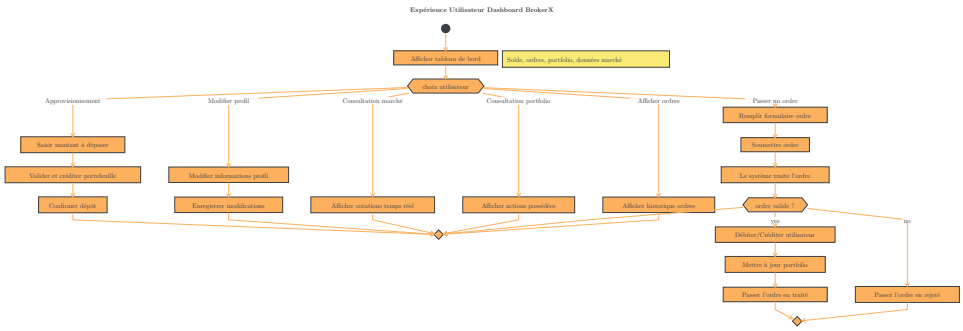


Fig. 2. – Dashboard Activity Diagram

Le système permet aux utilisateurs de :

- Gérer leur compte utilisateur
- Approvisionner leur compte virtuel

- Passer des ordres d'achat/vente
- Consulter l'état des ordres et le portefeuille
- Consulter les données du marché

4.1.1 Priorisation MoSCoW

- Must Have
  - UC-01 - Inscription & Vérification d'identité
  - UC-03 - Approvisionnement du portefeuille (dépôt virtuel)
  - UC-05 - Placement d'un ordre (marché/limite) avec contrôles pré-trade
- Should Have
  - UC-07 - Appariement interne & Exécution (matching)
  - UC-06 - Modification / Annulation d'un ordre
- Could Have
  - UC-08 - Confirmation d'exécution et Notifications
  - UC-02 - Authentification & MFA
  - UC-04 - Abonnement aux données de marché
- Won't Have

4.1.2 UC-01 - Inscription & Vérification d'identité

Ce cas d'utilisation est le point d'entrée d'un nouvel utilisateur sur la plateforme BrokerX. L'inscription comprend les étapes suivantes :

- Le client remplit un formulaire d'inscription avec des informations de base (email, mot de passe, etc).
- Le système crée son compte utilisateur avec le statut « Non vérifié »
- Il sera vérifié à la première connexion, et en attendant il ne pourra pas effectuer d'actions.

4.1.3 UC-03 - Approvisionnement du portefeuille (dépôt virtuel)

Ce cas d'utilisation permet à un utilisateur de déposer des fonds sur son compte BrokerX. Le processus comprend les étapes suivantes :

- L'utilisateur se connecte à son compte.
- Il navigue vers la section « Approvisionnement ».
- Il saisit le montant à déposer.
- Le système valide le montant (positif, limites).
- Le système crédite le portefeuille de l'utilisateur avec le montant déposé. (dépôt virtuel)
- Le système affiche confirmation

4.1.4 UC-05 - Placement d'un ordre (marché/limite) avec contrôles pré-trade

Ce cas d'utilisation permet à un utilisateur de passer un ordre d'achat ou de vente d'instruments financiers. Le processus comprend les étapes suivantes :

- L'utilisateur se connecte à son compte.
- Il navigue vers la section « Passer un ordre ».
- Il remplit le formulaire d'ordre avec les détails (type, quantité, prix, etc).
- Le système valide le formulaire (champs obligatoires, formats).
- La demande d'ordre est envoyée
- Le système effectue des contrôles pré-trade (solvabilité, limites de risque).
- L'utilisateur peut consulter l'état de son ordre en direct
- Le système traite l'ordre et met à jour son état (exécuté, rejeté, annulé).
- Le système notifie l'utilisateur du résultat de l'ordre.

4.1.5 UC-07 - Appariement interne & Exécution (matching)

Ce cas d'utilisation concerne l'exécution des ordres validés. Dans la version actuelle de BrokerX, il n'y a pas de véritable moteur de matching (appariement d'ordres d'achat et de vente). L'exécution est simulée par le MarketDataService :

- Le système reçoit un ordre validé via événement Redis
- Le MarketDataService simule l'exécution au prix du marché
- L'ordre passe de l'état VALIDATED à FILLED
- Les positions et soldes de l'utilisateur sont mis à jour
- Une notification d'exécution est envoyée

Note : Un vrai moteur de matching qui apparierait les ordres d'achat et de vente entre utilisateurs pourrait être implémenté dans une version future.

4.1.6 UC-06 - Modification / Annulation d'un ordre

Ce cas d'utilisation permet à un utilisateur de modifier ou d'annuler un ordre en attente. Pour chaque ordre en statut « Pending », l'utilisateur peut l'annuler ou le modifier (quantité, prix limite et type). Le système valide la demande et met à jour l'état de l'ordre en conséquence. (En réalité les ordres sont traités très rapidement donc cette fonctionnalité est difficile à utiliser)

4.1.7 UC-08 - Confirmation d'exécution et Notifications

Ce cas d'utilisation permet à un utilisateur de recevoir des confirmations d'exécution pour ses ordres passés. Lorsqu'un ordre est exécuté, le système envoie un email à l'utilisateur avec les détails de l'exécution (quantité, prix, frais, etc).

4.1.8 UC-02 - Authentification & MFA

Ce cas d'utilisation permet à un utilisateur de se connecter de manière sécurisée à son compte BrokerX en utilisant l'authentification multi-facteurs (MFA). Le processus comprend les étapes suivantes :

- L'utilisateur accède à la page de connexion.
- Il saisit son email et son mot de passe.
- Le système valide les informations d'identification.
- Si les informations sont correctes, le système envoie un code OTP par email à l'utilisateur.
- L'utilisateur saisit le code OTP reçu.
- Le système valide le code OTP.
- Le système transmet un token JWT pour les sessions authentifiées.

4.1.9 UC-04 - Abonnement aux données de marché

Ce cas d'utilisation permet aux utilisateurs de s'abonner à des symboles boursiers pour recevoir des rapports réguliers par email. Le processus comprend les étapes suivantes :

- L'utilisateur se connecte à son compte
- Il accède à la section « Abonnements »
- Il sélectionne un ou plusieurs symboles boursiers à suivre
- Le système enregistre les abonnements
- Le NotificationService envoie périodiquement des rapports avec les prix actuels et l'historique des symboles suivis

4.2 Contexte technique

- **Applications clientes** : Postman, frontend web
- **Couche API**: Axum (Rust) pour les endpoints RESTful

- **Couche base de données** : PostgreSQL
- **API Gateway** : Axum (Rust)
- **Communication** : Requêtes HTTP/HTTPS entre clients, gateway et microservices, Redis pour la communication événementielle

5 Stratégie de solution

Problème	**Approche de solution**
Performance	Utilisation du langage Rust
Gestion des données	Base relationnelle (**PostgreSQL**), cache en mémoire pour les évènements et les données fréquemment utilisées
Observabilité	Métriques Prometheus et Dashboards Grafana

Tableau 3. – Stratégie de solution

6 Domain-Driven Design (DDD)

6.1 Introduction au DDD

BrokerX applique les principes du Domain-Driven Design pour structurer le code métier autour des concepts du domaine financier. Cette approche permet une meilleure séparation des préoccupations et une évolutivité accrue du système.

6.2 Bounded Contexts

Le système BrokerX est divisé en plusieurs contextes délimités (Bounded Contexts), chacun représentant un sous-domaine métier distinct :

Bounded Context	**Responsabilité**	**Service**
User Context	Gestion des utilisateurs, authentification, soldes et positions	UserService
Order Context	Gestion du cycle de vie des ordres	OrderService
Market Data Context	Données de marché, cotations, exécution simulée	MarketDataService
Event Store Context	Event sourcing, saga chorégraphiée, outbox pattern transactionnel	EventStoreService
Notification Context	Notifications d'exécution d'ordres, alertes utilisateur	NotificationService

Tableau 4. – Bounded Contexts de BrokerX

Chaque contexte est implémenté comme un microservice autonome avec sa propre base de données, garantissant l'isolation et l'indépendance des domaines.

6.3 Entités et Value Objects

6.3.1 Entités principales

Les entités sont des objets métier identifiés par un identifiant unique (UUID) dont l'identité persiste dans le temps :

User Context

- `User` : Entité représentant un utilisateur du système

  - Attributs : `id` (UUID), `email`, `password_hash`, `balance`, `verified`, `created_at`
  - Invariants : email unique,
- `Holding` : Position d'un utilisateur sur un instrument
  - Attributs : `user_id`, `symbol`, `quantity`, `average_price`
  - Invariants : quantity > 0

Order Context
- `Order` : Entité centrale représentant un ordre de bourse
  - Attributs : `id` (UUID), `user_id`, `symbol`, `order_type`, `quantity`, `price`, `status`, `created_at`
  - Machine à états : `PENDING` → `VALIDATED` → `FILLED` / `REJECTED` / `CANCELLED`
  - Invariants : quantity > 0, price > 0 (pour ordres limites)

Market Data Context
- `Stock` : Instrument financier
  - Attributs : `symbol`, `name`, `current_price`, `last_updated`
  - Invariants : price > 0

6.3.2 Value Objects

Les Value Objects sont des objets immuables définis par leurs attributs :

- `OrderType` : Énumération `MARKET` | `LIMIT`
- `OrderStatus` : Énumération `PENDING` | `VALIDATED` | `FILLED` | `REJECTED` | `CANCELLED`
- `Side` : Énumération `BUY` | `SELL`
- `Money` : Représentation du montant avec précision (Decimal)

Event Objects (Value Objects spécialisés)
- `OrderEvent` : Événement immutable dans l'event store
- `OrderEventPayload` : Payload d'événement pour la communication Redis
- `OrderEventType` : Type d'événement dans la saga (`ORDER_CREATED`, `ORDER_VALIDATED`, etc.)

Les événements sont des Value Objects car ils sont :
- Immutables une fois créés (pas de modification après insertion)
- Définis entièrement par leur contenu (event_type, order_id, données métier)
- Utilisés pour la communication et l'audit, sans identité métier propre

6.4 Aggregates

Les aggregates définissent les frontières transactionnelles et de cohérence :

User Aggregate
- Racine : `User`
- Entités liées : `Holding[]`
- Règles : Toute modification du solde ou des positions doit passer par l'aggregate User pour garantir la cohérence

Order Aggregate
- Racine : `Order`
- Auto-suffisant : un ordre est traité de manière autonome
- Règles : Les transitions d'état sont contrôlées par la machine à états

6.5 Repositories

Le pattern Repository abstrait l'accès aux données et fournit une interface orientée collection :

```rust
trait UserRepository {
    fn find_by_id(&self, id: Uuid) -> Option<User>;
    fn find_by_email(&self, email: &str) -> Option<User>;
    fn save(&self, user: &User) -> Result<()>;
    fn update_balance(&self, user_id: Uuid, amount: Decimal) -> Result<()>;
}

trait OrderRepository {
    fn find_by_id(&self, id: Uuid) -> Option<Order>;
    fn find_by_user(&self, user_id: Uuid) -> Vec<Order>;
    fn save(&self, order: &Order) -> Result<()>;
    fn update_status(&self, id: Uuid, status: OrderStatus) -> Result<()>;
}
```

Chaque bounded context possède son propre repository isolé, implémenté avec PostgreSQL.

6.6 Domain Services

Les services de domaine encapsulent la logique métier qui ne correspond pas naturellement à une entité :

TradeValidator

- Responsabilité : Validation pré/post-trade des ordres
- Logique :
  - Vérification de solvabilité (solde suffisant pour achat)
  - Vérification des positions (quantité suffisante pour vente)
  - Contrôle des bandes de prix
- Collaboration : Interroge `UserRepository` et `MarketDataRepository`

ExecutionSimulator

- Responsabilité : Simulation de l'exécution sur le marché
- Logique : Interaction avec les autres services pour simuler l'exécution des ordres

6.7 Ubiquitous Language

Le **ubiquitous language** utilisé dans BrokerX reflète le vocabulaire métier de la finance :

Terme métier	**Signification dans le code**
Ordre	`Order` entity
Exécution	Passage d'un ordre de `VALIDATED` à `FILLED`
Position	`Holding` entity
Solde	`User.balance`
Pré/Post-trade	Validation avant/après exécution (`TradeValidator`)

Tableau 5. – Ubiquitous Language de BrokerX

6.8 Architecture en couches (Layered Architecture)

Bien que BrokerX utilise une architecture microservices, chaque service respecte une séparation en couches :

1. API Layer (HTTP Handlers) → Axum routes
2. Application Layer (Use Cases) → Orchestration
3. Domain Layer (Business Logic) → Entities, Services
4. Infrastructure Layer (Repos) → PostgreSQL, HTTP clients

Application Layer : Cas d'usage orchestrant les opérations du domaine

Infrastructure Layer : Implémentations concrètes des repositories et clients externes (mail par exemple)

Cette séparation permet de tester la logique métier indépendamment de l'infrastructure.

7 Vue des blocs de construction

7.1 Vue d'ensemble du système (Niveau 1)

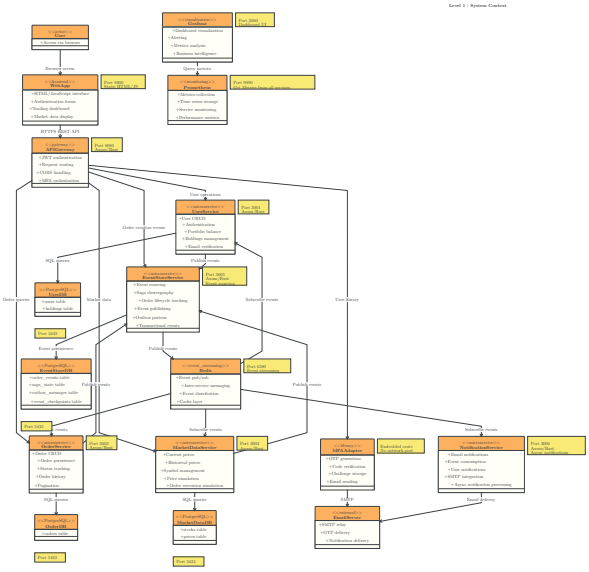


Fig. 3. – Vue d'ensemble du système

Le système BrokerX suit une architecture REST API avec passerelle API et microservices :

1. La passerelle API est le point d'entrée unique pour les clients.
2. La passerelle API orchestre l'authentification via MFA (→ token JWT)
3. Les différents services sont :
   - Service Utilisateurs : gestion des comptes et portefeuilles utilisateurs
   - Service Ordres : gestion des ordres en mémoire persistente
   - Service Données de Marché : récupération des cotations en temps réel, exécution des ordres sur le marché
   - Service Event Store : event sourcing, outbox pattern transactionnel
   - Service Notifications : envoi d'emails (Code OTP, confirmation d'exécution)
4. Prometheus collecte les métriques de tous les services
5. Grafana visualise les métriques via dashboards

7.2 Architecture du domaine (Niveau 2)

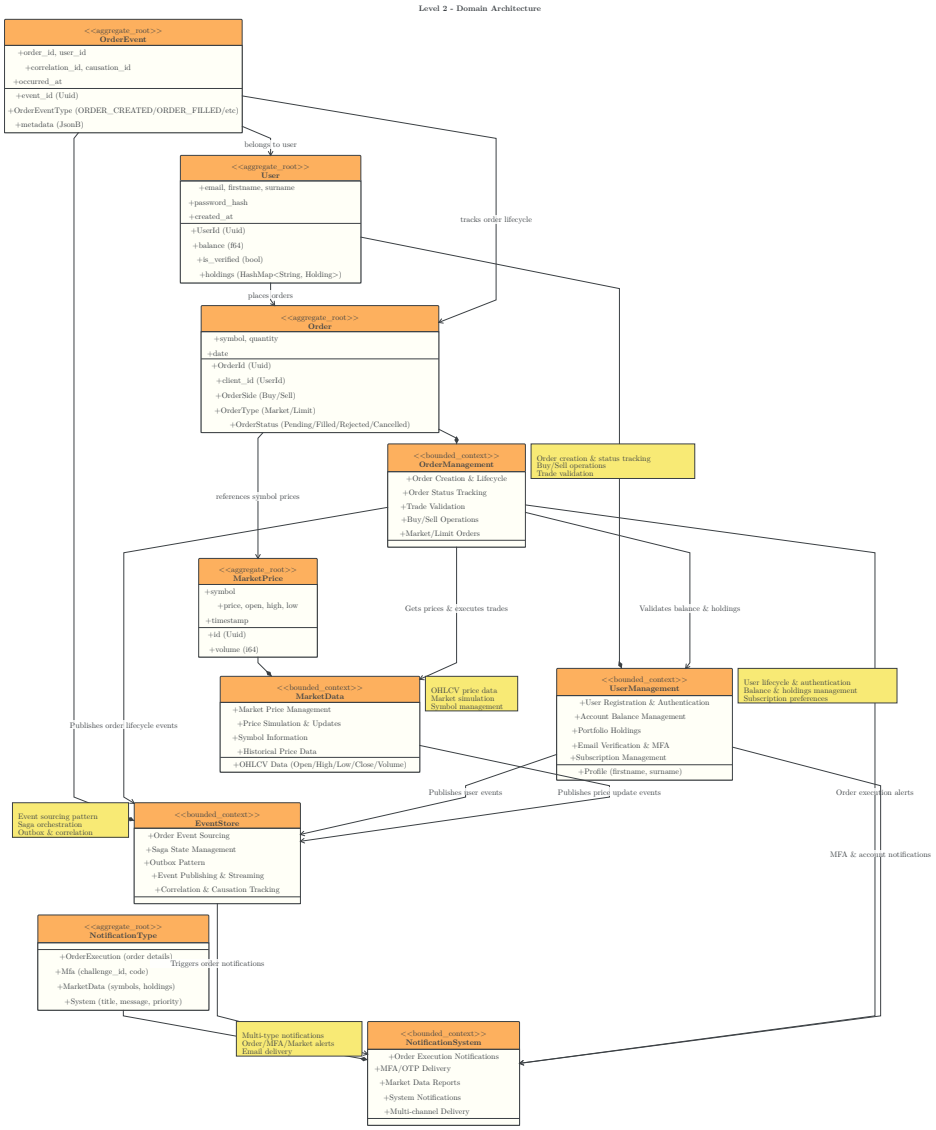


Fig. 4. – Architecture du domaine

Ce diagramme présente les 5 bounded contexts métier et leurs aggregate roots, avec les relations inter-domaines via événements pour maintenir la cohérence distribuée.

7.3 Architecture événementielle pour le traitement des ordres

Le système utilise une saga chorégraphiée distribuée pour traiter les ordres de manière asynchrone et résiliente. Le processus se décompose en deux phases :

1. **Acceptation synchrone** : L'API Gateway accepte l'ordre et retourne un `202 Accepted` dès que l'évènement `ORDER_CREATED` est publié dans l'Event Store
2. **Traitement asynchrone** : Une saga distribuée coordonne la validation, l'exécution et le règlement via événements Redis

7.4 Vue d'exécution

7.4.1 Parcours utilisateur complet

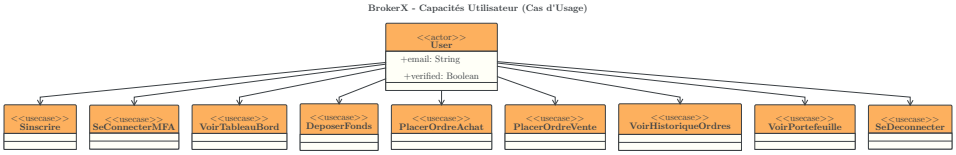


Fig. 5. – Parcours utilisateur complet

Note : Ce diagramme utilise la notation de classe pour représenter les cas d'usage (Pintora ne supportant pas nativement les diagrammes de cas d'usage).

Légende : `<<actor>>` *= Acteur,* `<<usecase>>` *= Cas d'usage, les flèches représentent les associations.*

Ce diagramme illustre l'ensemble des interactions utilisateur dans le système BrokerX, de l'inscription initiale jusqu'à la gestion complète du portefeuille. Il couvre tous les cas d'usage implémentés :

7.4.2 Flux principaux documentés

1. **Inscription et vérification d'identité (UC-01)**
   - Création de compte avec validation email
   - Processus de vérification par code à 6 chiffres
   - Activation du compte utilisateur
2. **Authentification et MFA (UC-02)**
   - Connexion sécurisée avec email/mot de passe
   - Authentification multi-facteurs obligatoire
   - Génération et validation de tokens JWT
3. **Approvisionnement du portefeuille (UC-03)**
   - Dépôt virtuel de fonds
4. **Placement d'ordres avec contrôles pré-trade (UC-05)**
   - Validation des formulaires d'ordre
   - Contrôles de risque et de solvabilité
   - Traitement asynchrone par le service d'ordres
5. **Modification et annulation d'ordres (UC-06)**
   - Modification des ordres en statut PENDING (quantité, prix limite, type)
   - Annulation immédiate des ordres non encore exécutés
   - Validation des autorisations utilisateur
6. **Appariement interne et exécution (UC-07)**
   - Simulation de l'exécution sur le marché via MarketDataService
   - Mise à jour temps réel des statuts d'ordre (PENDING → VALIDATED → FILLED)
7. **Confirmation d'exécution et notifications (UC-08)**
   - Envoi automatique d'emails de confirmation post-exécution
   - Notifications incluant détails complets (quantité, prix, frais, nouveau solde)

- Traçabilité des tentatives d'envoi et statuts de livraison

8. **Consultation des données de marché (UC-04)**

- Accès aux cotations en temps réel des instruments financiers
- Historique des prix et données de marché
- Mise à jour périodique des prix (cycle de 5 secondes)

7.5 Vue de déploiement

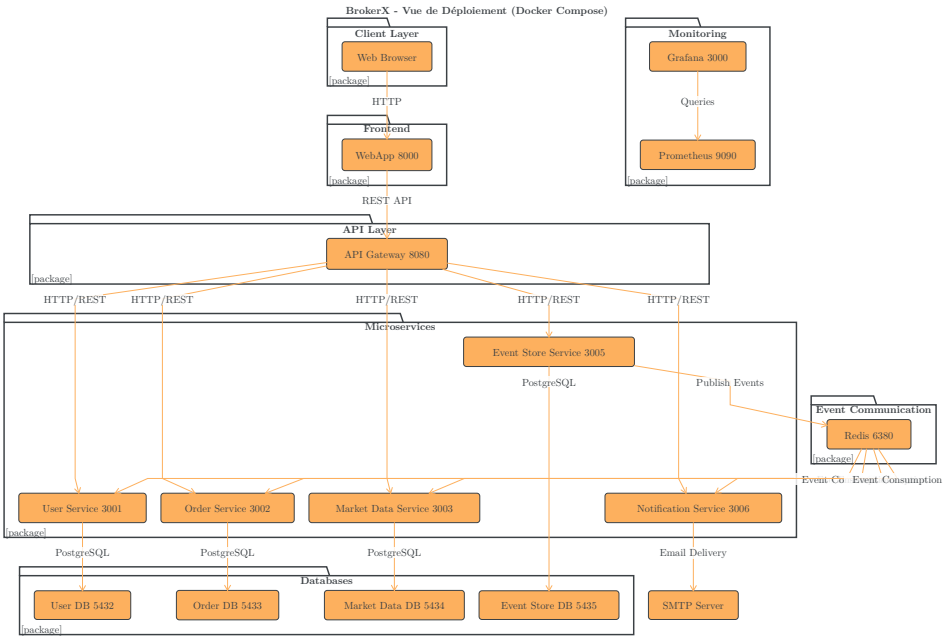


Fig. 6. – Vue de déploiement

L'architecture de déploiement utilise **Docker Compose** pour orchestrer l'ensemble des composants en conteneurs. La stack complète comprend 6 microservices, 4 bases de données PostgreSQL, Redis comme message broker, et une stack de monitoring complète.

Le diagramme met en évidence la communication événementielle via Redis ainsi que les connexions directes API Gateway vers microservices.

Les requêtes Prometheus → `/metrics` (de chaque service) ne sont pas représentées par souci de lisibilité.

7.5.1 Couche Frontend

- **WebApp 8000** : Interface web statique servie via serveur HTTP, accessible par les navigateurs clients

7.5.2 Couche API

- **API Gateway** (port 8080) : Point d'entrée unique orchestrant toutes les requêtes clients vers les microservices via HTTP/REST

7.5.3 Couche Microservices

- **User Service** (port 3001) : Gestion des utilisateurs et authentification, consomme les événements Redis pour mise à jour des soldes
- **Order Service** (port 3002) : Gestion des ordres avec event consumer Redis pour traitement des validations

- **Market Data Service** (port 3003) : Données de marché et simulation d'exécution, consomme les événements pour déclencher les exécutions
- **Event Store Service** (port 3005) : Event sourcing central, publie tous les événements vers Redis via transactional outbox
- **Notification Service** (port 3006) : Envoi d'alertes et notifications par email, connecté au serveur SMTP externe

7.5.4 Couche Données

Chaque service possède sa propre base de données PostgreSQL 15 pour l'isolation :

- **User DB** (port 5432) : Données utilisateurs et holdings
- **Order DB** (port 5433) : Ordres avec table `processed_events` pour idempotence
- **Market Data DB** (port 5434) : Instruments financiers et historique des prix
- **Event Store DB** (port 5435) : Tables `order_events`, `saga_state`, `outbox` pour event sourcing

7.5.5 Communication Événementielle

- **Redis** (port 6380) : Message broker central recevant les événements de l'EventStoreService et les distribuant aux event consumers de chaque microservice
- **Event Flow** : EventStoreService → Redis → (OrderService, MarketDataService, UserService, NotificationService)
- **Transactional Outbox Worker** : Traitement périodique de l'outbox dans EventStoreService (polling 100ms)
- **Event Consumers** : Abonnements Redis dans chaque service pour saga chorégraphiée distribuée

7.5.6 Monitoring

- **Prometheus** (port 9090) : Collecte des métriques via endpoints `/metrics` de tous les microservices
- **Grafana** (port 3000) : Visualisation des métriques via requêtes Prometheus et dashboards pré-configurés
- **K6** : Des scénarios de tests de charge sont disponibles

7.5.7 Services Externes

- **SMTP Server** : Serveur de messagerie externe pour l'envoi des emails MFA et notifications d'exécution

7.5.8 Réseau et Communication

- **Réseau Docker** : Tous les services communiquent via un réseau privé `brokerx-network`
- **Communication synchrone** : Client → WebApp → API Gateway → Microservices (HTTP/REST)
- **Communication asynchrone** : EventStoreService → Redis → Event Consumers (saga événementielle)
- **Connexions BDD** : Chaque microservice avec sa base PostgreSQL dédiée

8 Concepts transversaux

8.1 Décisions d'Architecture

Voir Chapitre 12

8.2 Exigences qualité

BrokerX est capable de gérer $\geq$ 1200 ordres/s en charge continue en maintenant une latence P95 <= 100 ms sur l'endpoint `POST /v1/orders` de l'API Gateway. Voir ci-dessous une capture d'écran du tableau de bord Grafana pendant un test de charge.

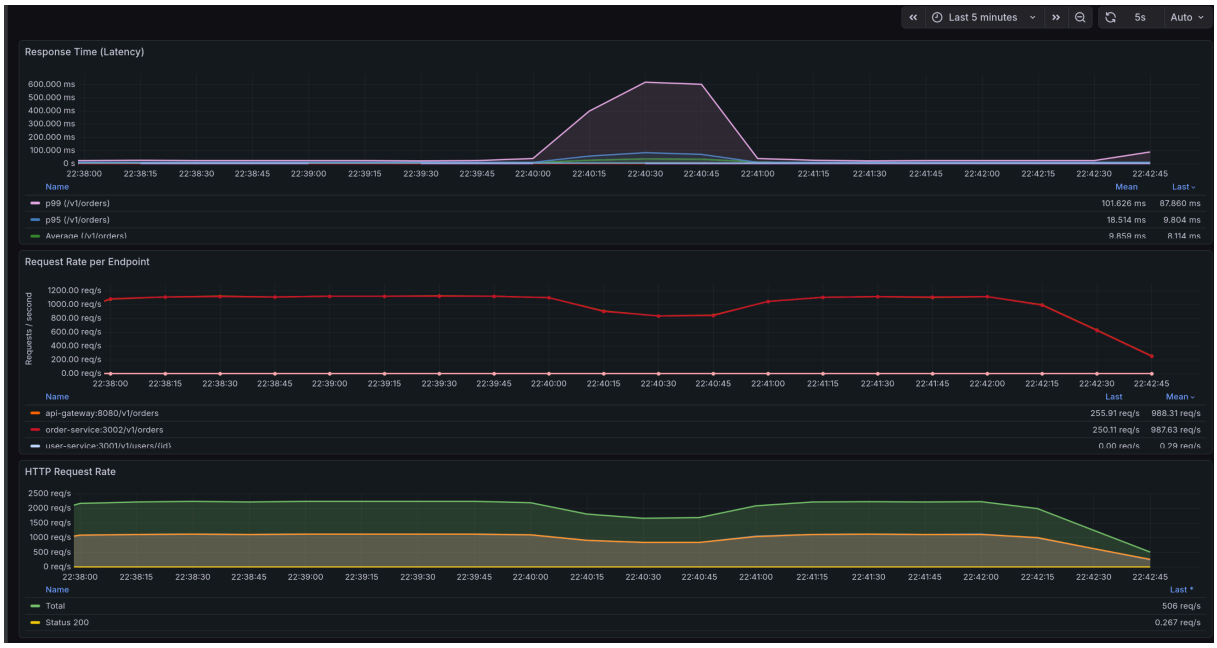


Fig. 7. – Vue Grafana

Les tests de charges extrêmes montrent que le système est capable de maintenir 5000 orders/s sur une courte période sans que la latence P95 de l'endpoint `POST /v1/orders` ne dépasse 100 ms.



Fig. 8. – Vue Grafana 5000 ordres/s

8.3 CI/CD (Intégration et Déploiement Continu)

BrokerX utilise **GitHub Actions** pour l'automatisation CI/CD avec trois pipelines principaux :

Pipeline	Déclencheur	Actions
CI	Push sur `main` et PRs	Tests + formatage Rust
Release	Tags `v*.*.*`	Build multi-plateformes via `cargo-dist`
Bump	Manuel	Incrémentation de versions

Tableau 6. – Pipelines CI/CD

9 Runbook

Le runbook suivant documente les procédures opérationnelles pour le déploiement de BrokerX.

9.1 Vue d'ensemble des services

BrokerX est composé de 6 microservices, 4 bases de données PostgreSQL, Redis et une stack de monitoring (Prometheus + Grafana).

Service	Port	URL
Web App	8000	`http://localhost:8000`
API Gateway	8080	`http://localhost:8080`
User Service	3001	`http://localhost:3001`
Order Service	3002	`http://localhost:3002`
Market Data Service	3003	`http://localhost:3003`
Event Store Service	3005	`http://localhost:3005`
Notification Service	3006	`http://localhost:3006`
Redis	6380	Port 6379 interne
Prometheus	9090	`http://localhost:9090`
Grafana	3000	`http://localhost:3000`

Tableau 7. – Services BrokerX et ports associés

9.2 Déploiement

9.2.1 Prérequis

- Docker Engine 20.10+
- Docker Compose 2.0+
- Just (optionnel, recommandé)

9.2.2 Démarrage rapide

Avec Just :

```
just up
```

Avec Docker Compose :

```
docker compose up -d
```

Les services seront accessibles sur :

- Web App : `http://localhost:8000`
- API Gateway : `http://localhost:8080`
- Swagger UI : `http://localhost:8080/swagger-ui/`
- Grafana : `http://localhost:3000` (admin/admin123)
- Prometheus : `http://localhost:9090`

9.2.3 Commandes utiles

```
# Arrêter tous les services
just down
# ou
docker compose down

# Voir les logs
just logs
# ou
docker compose logs -f

# Reconstruire les services
just rebuild
# ou
docker compose up -d --build

# Nettoyer complètement (supprime les volumes)
just clean
# ou
docker compose down -v
```

9.3 Surveillance et monitoring

9.3.1 Grafana

URL : `http://localhost:3000`

Identifiants : admin / admin123

Dashboard : Importer `./grafana_dashboard.json` via Dashboards → Import

9.3.2 Prometheus

URL : `http://localhost:9090`

Métriques exposées par chaque service sur `/metrics`.

9.3.3 Logs

```
# Tous les services
docker compose logs -f

# Service spécifique
docker compose logs -f api-gateway
docker compose logs -f order-service

# Filtrer par niveau
docker compose logs -f | grep ERROR
```

Niveaux de logs : ERROR, WARN, INFO, DEBUG, TRACE

9.4 Tests de charge

Les tests K6 permettent de valider les performances du système.

9.4.1 Configuration

Activer le bypass MFA : Éditer `docker-compose.yml`, section `api-gateway` et `notification-service` :

```
environment:
  - MFA_BYPASS=true
```

Redémarrer : `docker compose up -d api-gateway`

9.4.2 Exécution

```
# Lancer les tests
just load-test quality-validation
# ou
cd load-tests && k6 run scenarios/quality-validation.js
```

Scénarios disponibles :

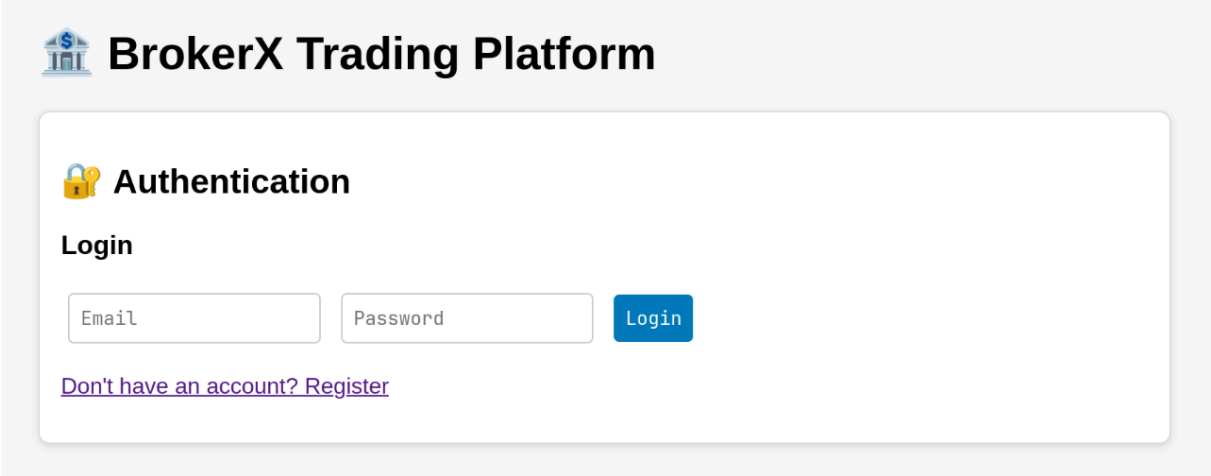
- `quality-validation.js` : Test de charge complet (≥1200 ordres/s)
- `market-data-consultation.js` : Test de consultation des données de marché

10 Guide d'utilisation

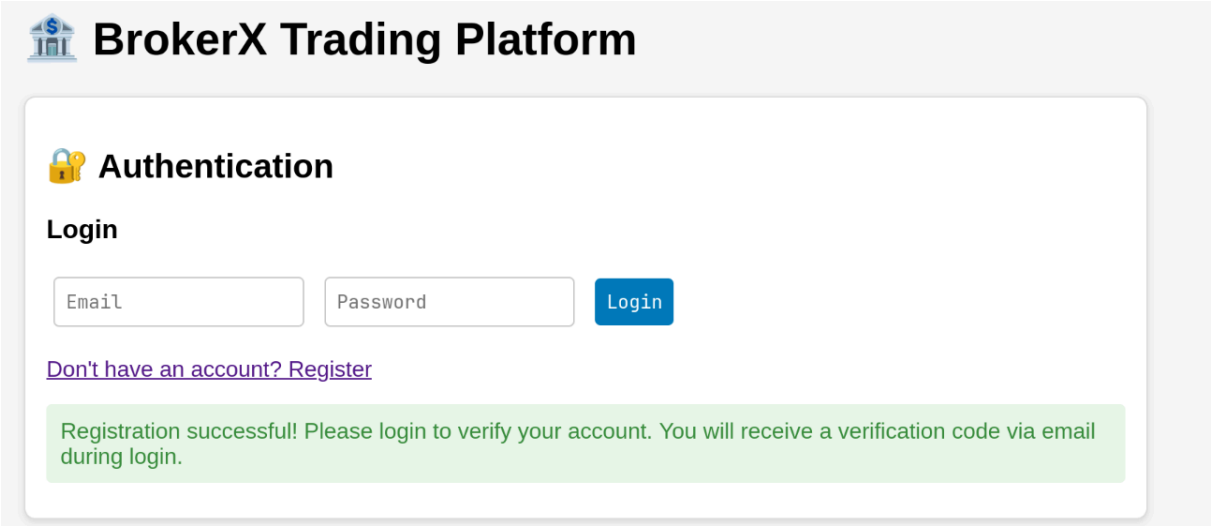
10.1 Frontend Web

Créer un nouveau compte utilisateur:

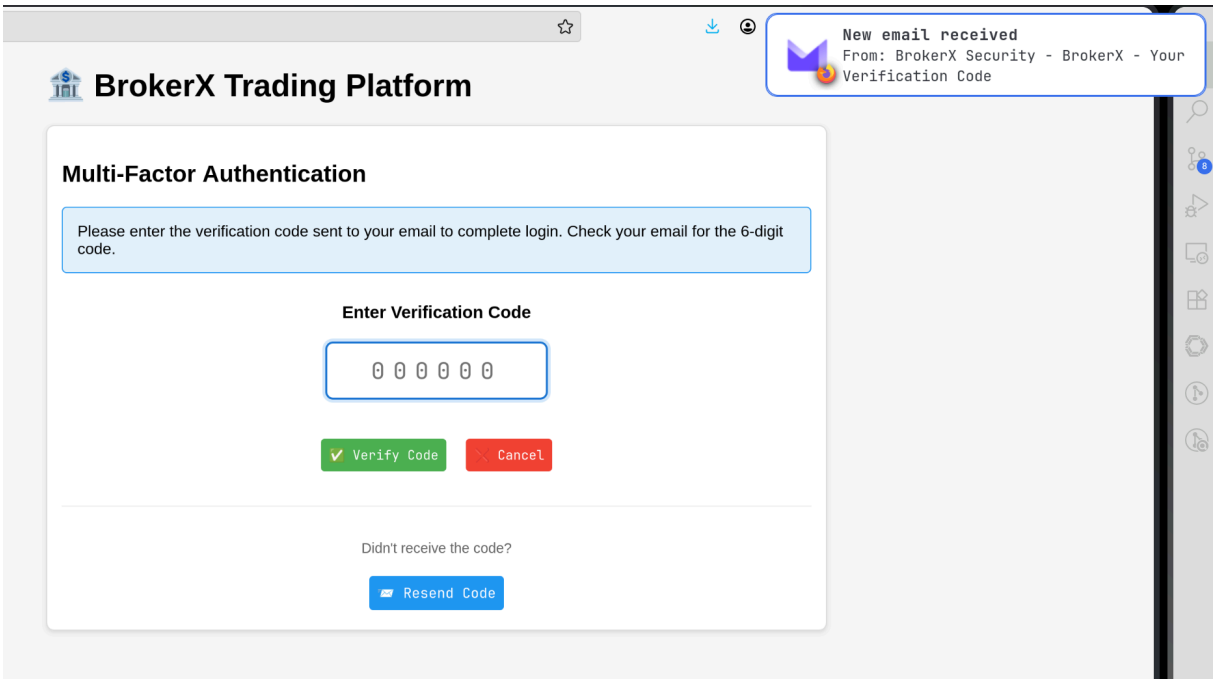
On peut cliquer sur « Register »:



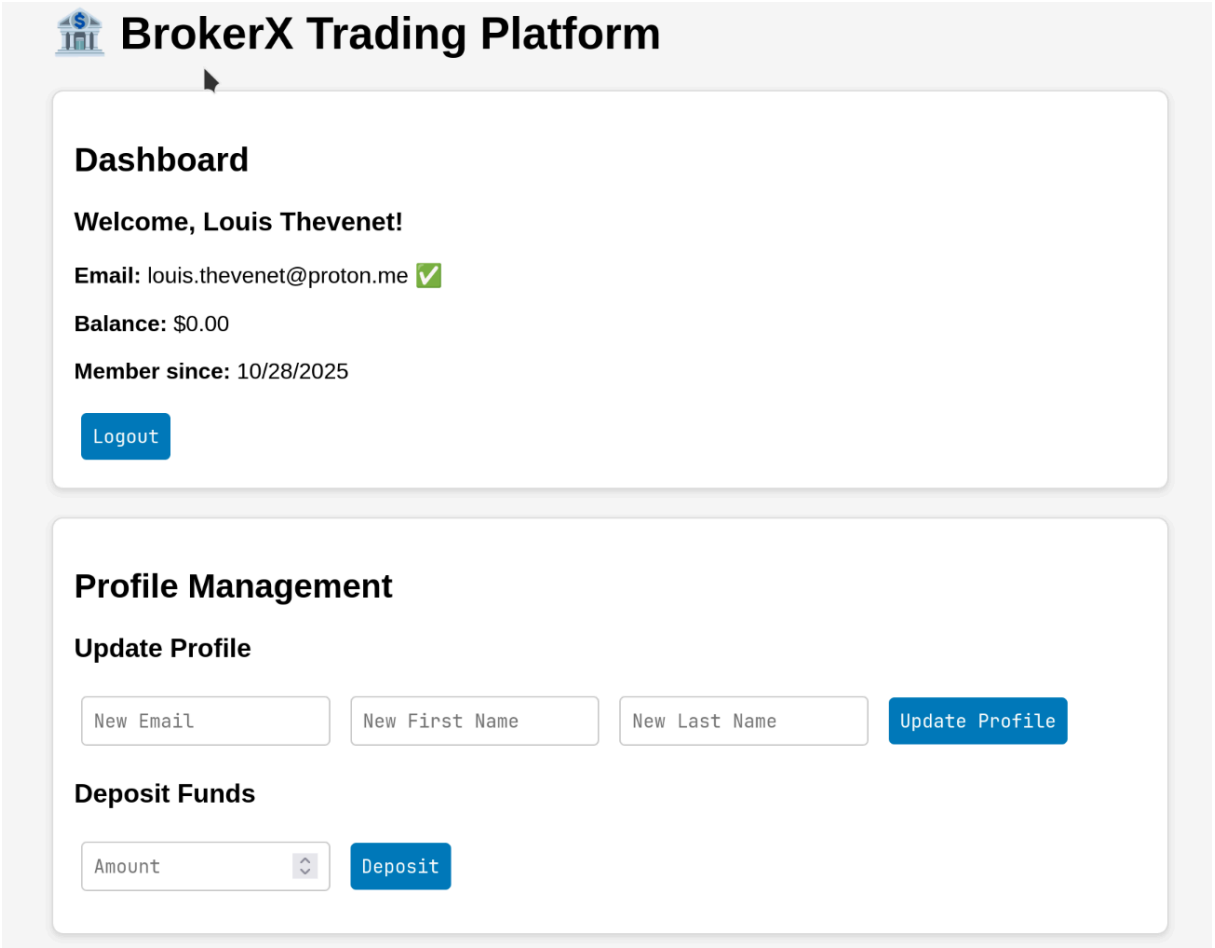
Après avoir rempli le formulaire d'inscription:



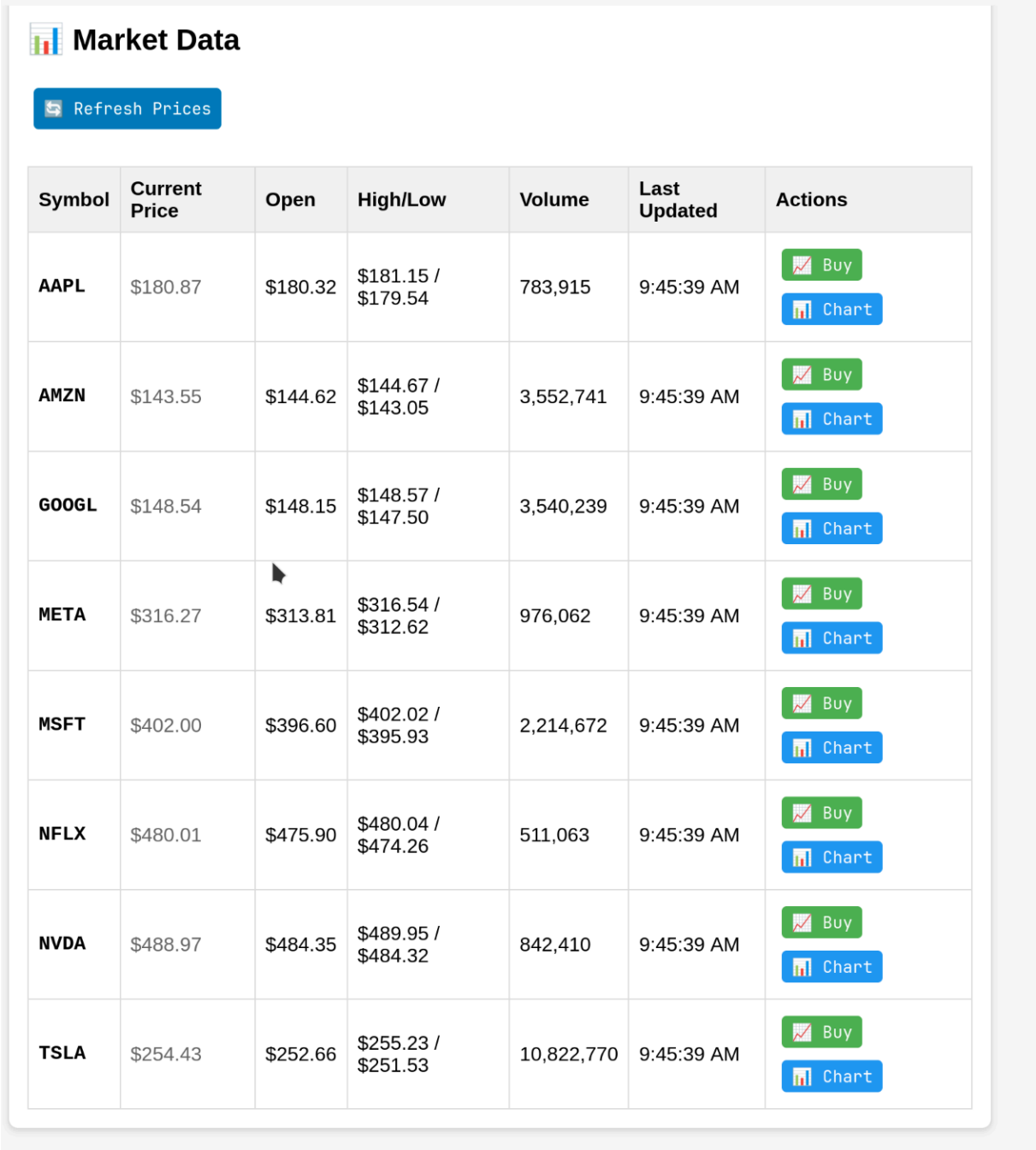
On peut maintenant se connecter:



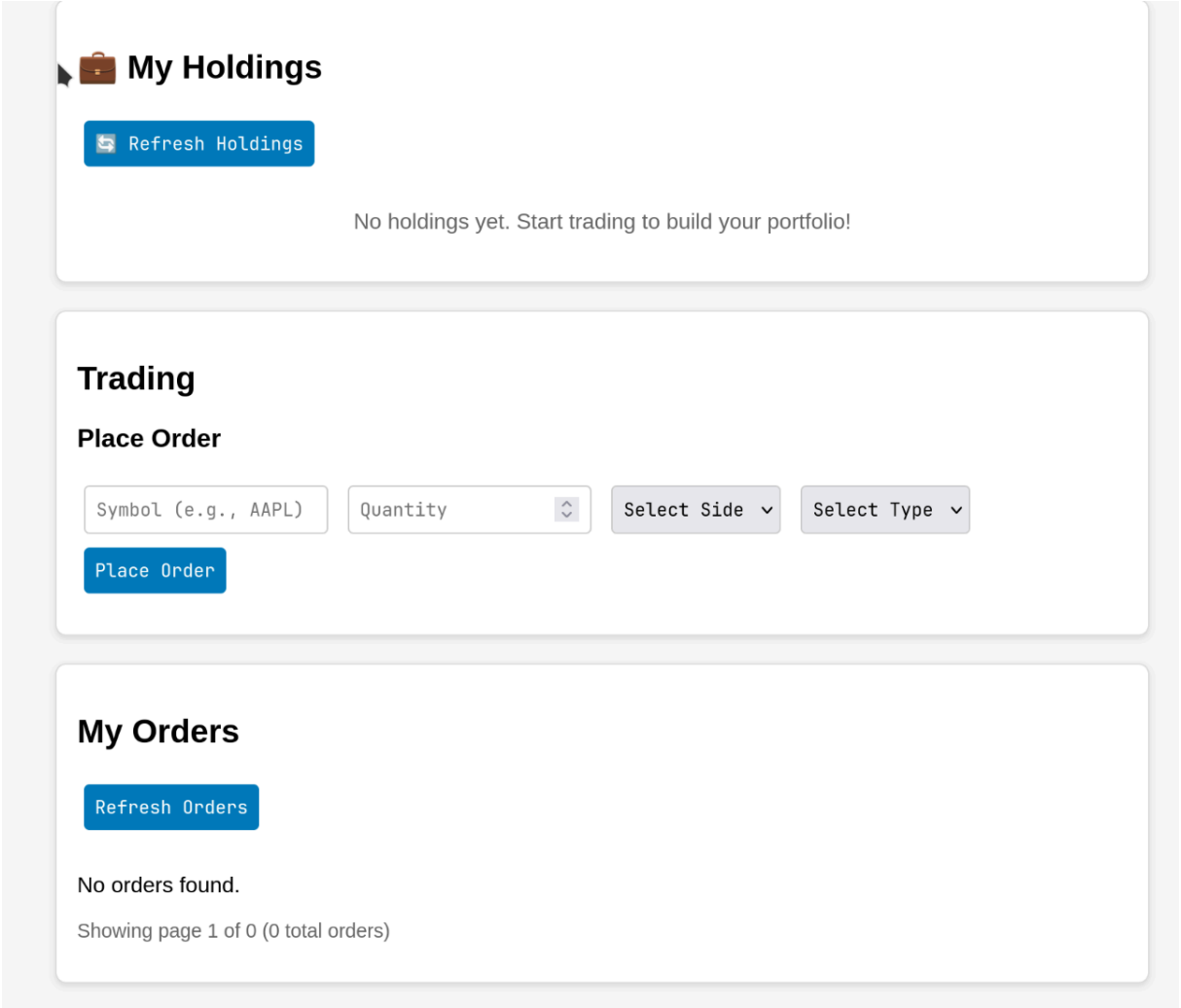
Vue informations personnelles:



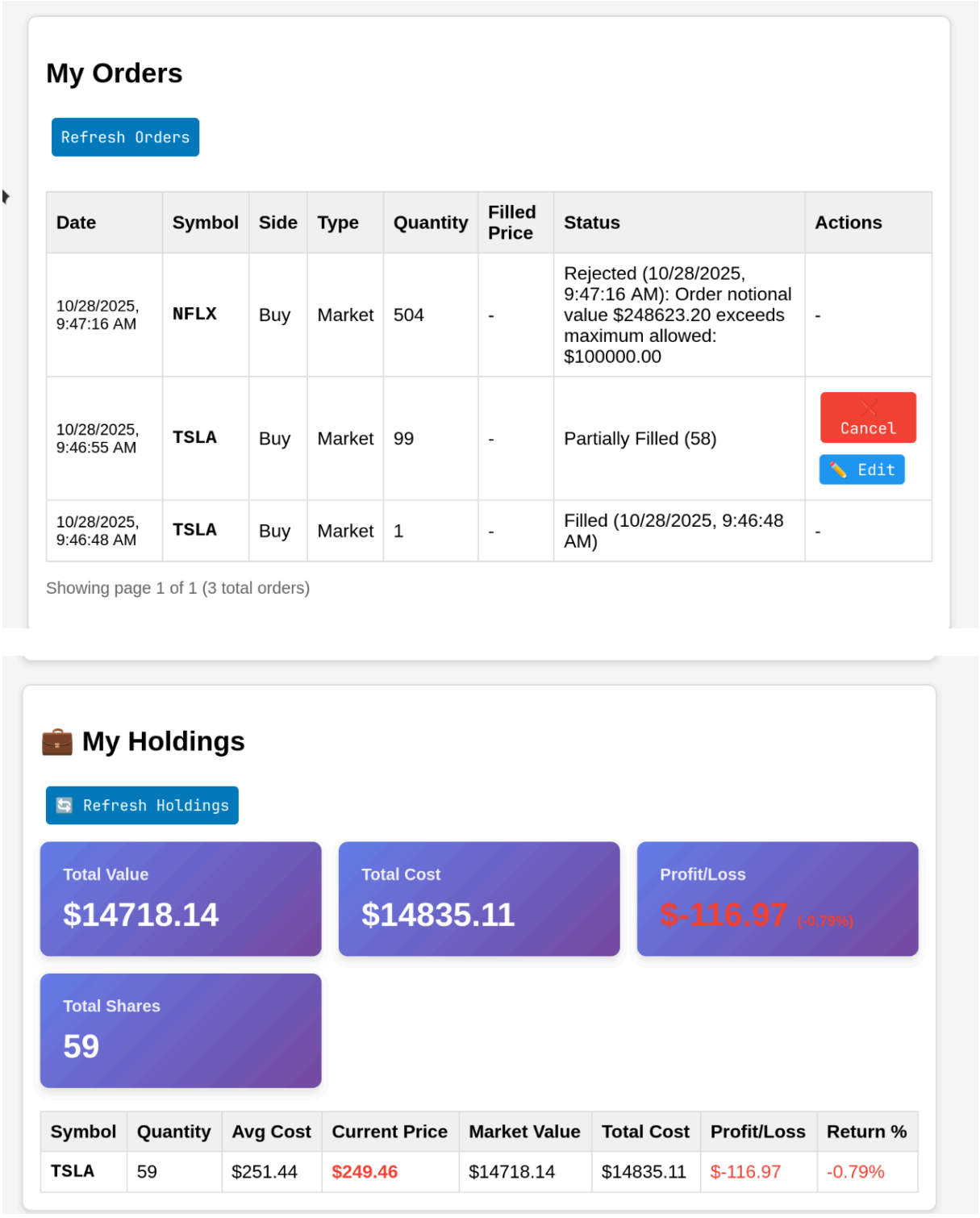
Vue marché:



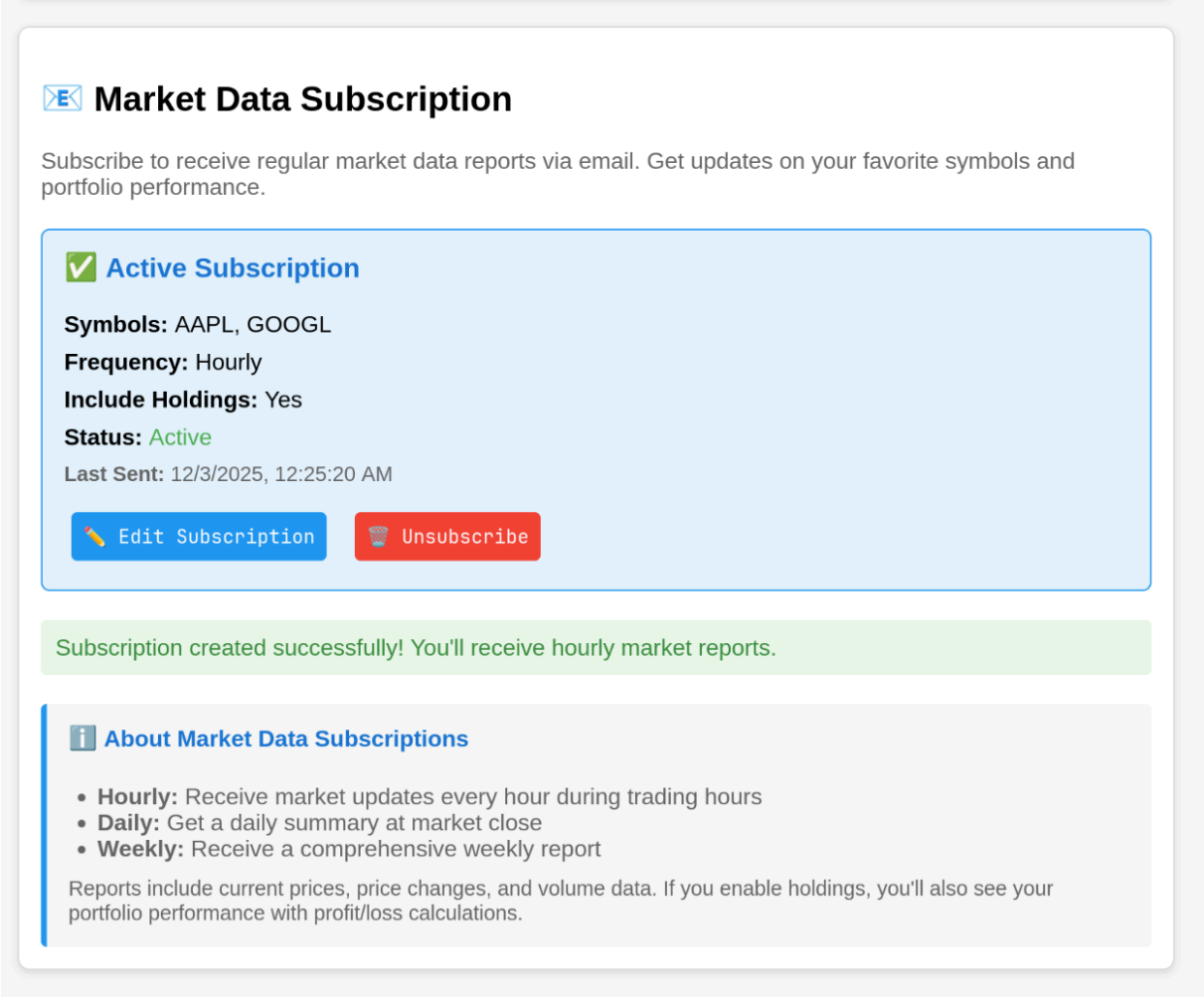
Portfolio, ordres passés et formulaire de passage d'ordre:



Après avoir passé des ordres:



On peut également s'abonner à des symboles et recevoir des rapports réguliers:



10.2 API

10.2.1 Swagger UI

La documentation est générée automatiquement via OpenAPI/Swagger et disponible aux endpoints suivant :

API Gateway http://127.0.0.1:8080/swagger-ui/
User Service http://127.0.0.1:3001/swagger-ui/
Order Service http://127.0.0.1:3002/swagger-ui/
Market Data Service http://127.0.0.1:3003/swagger-ui/
Event Store Service http://127.0.0.1:3005/swagger-ui/

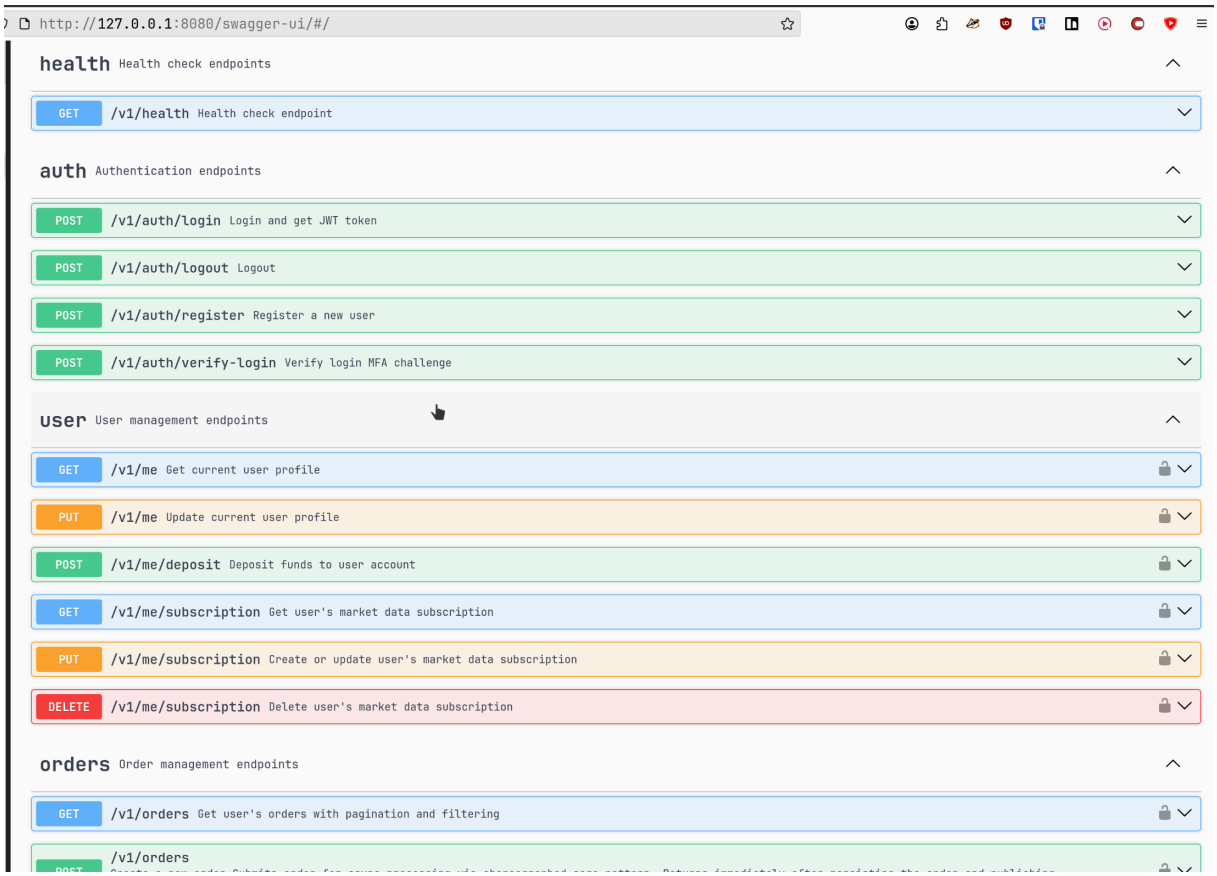


Fig. 9. – Swagger UI pour la paserelle API

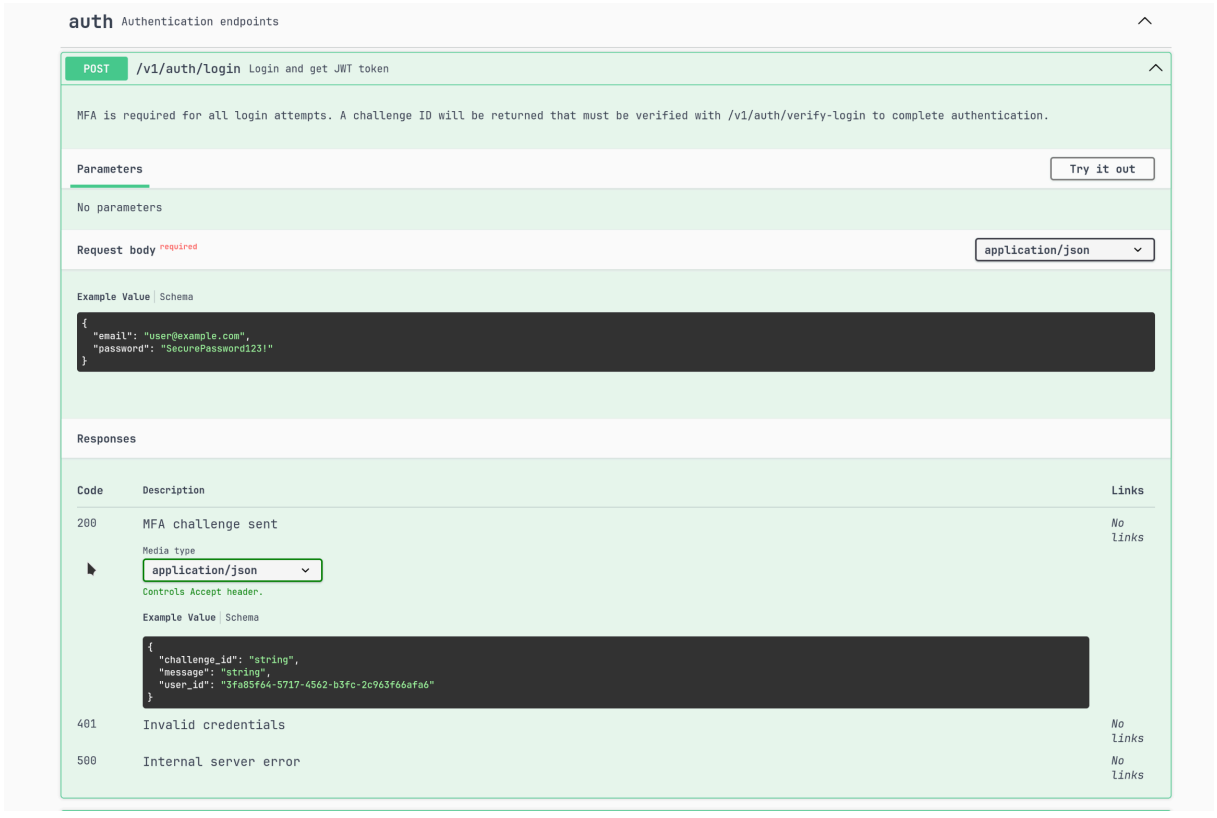


Fig. 10. – Détail sur l'endpoint `/v1/auth/login`

10.2.2 Postman

Les collections sont également disponibles dans le répertoire `docs/postman/` ou aux endpoints suivant:

API Gateway http://127.0.0.1:8080/api-docs/openapi.json
User Service http://127.0.0.1:3001/api-docs/openapi.json
Order Service http://127.0.0.1:3002/api-docs/openapi.json
Market Data Service http://127.0.0.1:3003/api-docs/openapi.json
Event Store Service http://127.0.0.1:3005/api-docs/openapi.json

11 Glossaire

Terme	Définition
Ordre	Instruction d'achat ou de vente d'un instrument financier
Portefeuille	Ensemble des actifs financiers détenus par un utilisateur
MFA	Authentification multi-facteurs
OTP	One-Time Password, mot de passe à usage unique
JWT	JSON Web Token, format de token sécurisé pour l'authentification

12 Annexe

12.1 Décisions d'architecture

12.1.1 Choix de l'architecture web et frontend (API Gateway + SPA)

Statut Acceptée

Contexte Pour l'implémentation de l'interface web client de BrokerX, nous devions choisir une architecture appropriée. L'application doit fournir :

- Une interface d'authentification (login/registration) sécurisée avec MFA
- Un tableau de bord client avec informations de compte en temps réel
- Un système de placement d'ordres interactif
- Une consultation des données de marché en temps réel
- Une architecture extensible pour futures fonctionnalités
- Des performances élevées pour traiter les ordres en temps réel

Décision Nous avons choisi une **architecture avec API Gateway et frontend SPA (Single Page Application)**

Architecture implémentée

Frontend

- **Single Page Application** (SPA) en HTML/CSS/JavaScript vanilla
- Interface avec navigation côté client
- Communication via API REST uniquement

Backend

API Gateway (Axum) comme point d'entrée unique
Authentification JWT + MFA obligatoire (codes OTP par email)
Microservices : User Service, Order Service, Market Data Service

Conséquences

Positives

Séparation des responsabilités Frontend découplé du backend

Sécurité JWT + MFA obligatoire pour tous les utilisateurs

Performance SPA avec cache client et requêtes asynchrones

Extensibilité API REST documentée avec OpenAPI/Swagger

Négatives

Latence réseau Dépendance aux appels API pour toutes les interactions

Sécurité client Token JWT stocké dans localStorage (vulnérable au XSS)

12.1.2 Adoption des UUIDs comme identifiants pour clients et ordres

Statut Acceptée

Contexte Dans le cadre du développement de BrokerX, nous devons choisir un format d'identifiants pour les entités principales : clients et ordres. Ces identifiants doivent être uniques, non prévisibles, et facilement générables.

Décision Nous utiliserons des UUIDs (Universally Unique Identifiers) pour identifier les clients et les ordres dans le système. Les UUIDs ne garantissent pas strictement la propriété d'unicité, il est cependant très peu probable d'avoir des collisions, ils sont largement utilisés dans l'industrie pour des cas similaires.

Conséquences Nous remplacerons les types ID par un type UUID dans les structures de données concernées.

12.1.3 Choix de la technologie de logs (tracing pour journaux structurés)

Statut Acceptée

Contexte

Le domaine métier de BrokerX nécessite un système de journalisation immuable pour garantir la traçabilité des opérations critiques.

Options considérées

1. `log` : La crate `log` standard de Rust, simple et largement adoptée.
2. `slog` : Crate de logging structuré avec une approche hiérarchique et performante.
3. `tracing` : Crate moderne offrant un logging structuré avec support pour le tracing distribué et les spans.
4. `env_logger` : Logger simple basé sur les variables d'environnement, souvent utilisé avec la crate `log`.

Décision Nous avons choisi la crate `tracing` de Rust pour la gestion des journaux de logs. Celle-ci se marie bien avec nos dépendances actuelles (axum) et nous permettra d'implémenter une solution robuste et performante.

Conséquences Les journaux de logs seront centralisés et accessibles en temps réel. Nous devrons choisir où et quand inscrire de nouvelles entrées et quel niveau de log utiliser (info, debug, error, etc.).

12.1.4 Implémentation du traitement des ordres avec saga chorégraphiée

Statut Acceptée

Contexte Nous devons traiter les ordres de trading de manière asynchrone et fiable dans un environnement distribué. Les exigences sont :

Traitement asynchrone Accepter immédiatement les ordres, traiter en arrière-plan
Cohérence distribuée Coordonner validation, exécution et settlement entre services
Résilience Gérer les pannes et permettre la compensation en cas d'échec
Observabilité Tracer l'état de chaque ordre dans le pipeline

Décision Nous avons implémenté une **saga chorégraphiée** basée sur des événements avec Event Store.

Architecture de la saga **Processus en 2 phases** :

1. **Acceptation synchrone** : L'API Gateway persiste l'ordre dans Order Service et répond `202 Accepted`
2. **Traitement asynchrone** : Les services réagissent aux événements Redis de manière autonome

Étapes de la saga

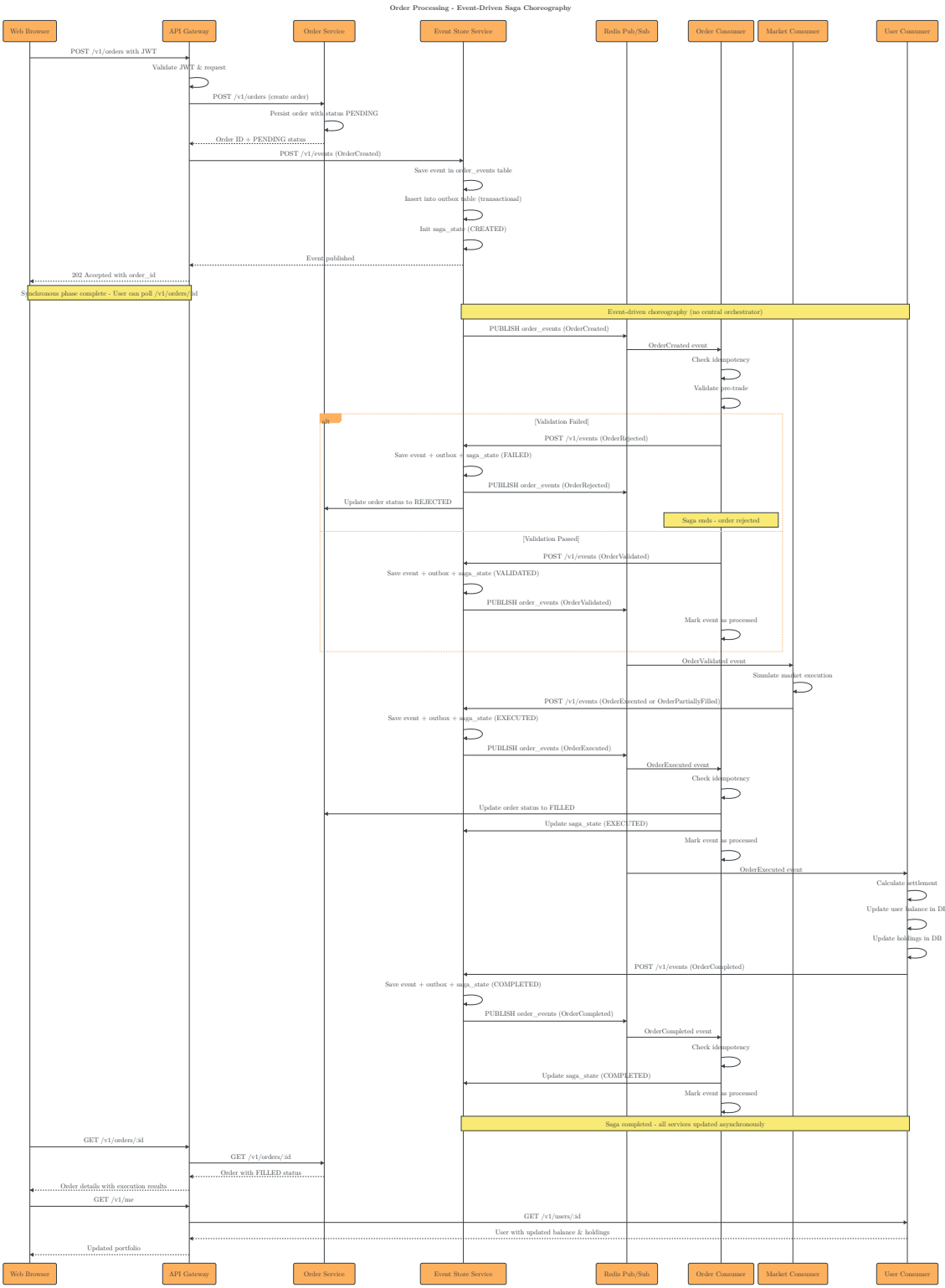


Fig. 11. – Détail du traitement des ordres avec saga chorégraphiée

Chorégraphie (chaque service réagit aux événements)

1. **API Gateway** -> publie `OrderCreated` dans Event Store
2. **Order Service (consommateur)**
   - Écoute `OrderCreated` -> valide pré-trade -> publie `OrderValidated` ou `OrderRejected`
   - Écoute `OrderExecuted` -> met à jour statut ordre -> `Filled` / `PartiallyFilled`
3. **Market Data Service (consommateur)**

- Écoute `OrderValidated` -> simule exécution -> publie `OrderExecuted` / `OrderPartiallyFilled`

4. **User Service (consommateur)**
   - Écoute `OrderExecuted` -> met à jour balance et holdings -> publie `OrderCompleted`
5. **Event Store Service**
   - Persiste tous les événements
   - Traque l'état de la saga (`saga_state`)
   - Outbox pattern pour publication fiable

Pas d'orchestrateur central : Chaque service décide de ses actions basé sur les événements qu'il consomme.

Mécanismes clés

Idempotence Chaque événement est marqué comme traité (table `processed_events`) pour éviter le double-traitement

Outbox Pattern Les événements sont persistés transactionnellement avec leurs effets, puis publiés de manière asynchrone

État de saga Table `saga_state` dans Event Store pour tracking et debugging (`CREATED` -> `VALIDATED` -> `EXECUTED` -> `COMPLETED`)

Conséquences

Avantages

Découplage total Aucune dépendance directe entre services (seulement événements)
Résilience Chaque service peut redémarrer indépendamment et reprendre la consommation
Traçabilité Event Store avec historique complet et rejouable de chaque ordre
Idempotence Protection contre événements dupliqués via `processed_events`
Scalabilité horizontale Chaque consommateur peut être répliqué

Inconvénients

Complexité Le flux est distribué sur plusieurs services, plus difficile à déboguer
Cohérence éventuelle L'état final est cohérent mais pas instantané
Dépendance Redis Point de défaillance unique pour la communication

Implémentation technique

Event Store PostgreSQL avec `order_events`, `saga_state`, `outbox`, `processed_events`
Communication Redis pub/sub via `brokerx-events` crate
Consommateurs `EventSubscriber` dans chaque service écoute le channel `order_events`
Pattern Outbox Publication fiable des événements depuis Event Store vers Redis

12.1.5 Architecture microservices (5 services distincts + API Gateway)

Statut Acceptée

Contexte Le système BrokerX doit gérer des volumes élevés d'ordres avec des responsabilités distinctes : gestion des utilisateurs, persistance des ordres, simulation de marché, et orchestration. Une architecture monolithique rendrait difficile le scaling indépendant et l'évolution de chaque composant.

Décision

Architecture microservices avec **5 services distincts** communiquant via HTTP/REST et événements Redis pour la coordination asynchrone de certaines tâches.

Les services

1. API Gateway (`:8080`)

   Point d'entrée unique pour les clients.

   - Authentification JWT et MFA
   - Routage vers les services appropriés
   - Pas de base de données (stateless)

2. User Service (`:3001`)

   Gestion des utilisateurs et de leurs comptes.

   - Création d'utilisateurs et authentification
   - Gestion des balances et holdings (positions)
   - Base de données : PostgreSQL `brokerx_users` (port 5432)

3. Order Service (`:3002`)

   Persistance et historique des ordres.

   - CRUD des ordres
   - Stockage des états (Pending, Filled, Rejected, Cancelled)
   - Base de données : PostgreSQL `brokerx_orders` (port 5433)

4. Market Data Service (`:3003`)

   Simulation du marché financier.

   - Génération et mise à jour des prix (toutes les 5s)
   - Exécution simulée des ordres via event consumer
   - Base de données : PostgreSQL `brokerx_market_data` (port 5434)

5. Event Store Service (`:3005`)

   Event sourcing et coordination de la saga.

   - Persistance de tous les événements d'ordres
   - État de la saga (saga_state)
   - Outbox pattern pour publication fiable vers Redis
   - Base de données : PostgreSQL `brokerx_event_store` (port 5435)

Infrastructure

Redis (`:6380`) Pub/Sub pour la communication événementielle entre services
Notification Service Envoi d'emails (MFA, notifications d'ordres)

Coordination des services

Communication synchrone

- API Gateway → Services métier (User, Order, Market Data, Event Store)
- Requêtes HTTP/REST avec clients optimisés

Communication asynchrone (Saga chorégraphiée)

- Event Store publie les événements dans Redis (channel `order_events`)
- Chaque service écoute les événements via `EventSubscriber`

- Pattern Outbox pour publication fiable

Conséquences

Avantages

Scalabilité indépendante Chaque service peut être scalé selon ses besoins

Isolation des pannes Un service en panne n'affecte pas les autres

Déploiements indépendants Mise à jour sans tout redémarrer

Séparation claire Chaque service a une responsabilité unique

Découplage événementiel Les services ne se connaissent que via événements ou API

Traçabilité Event Store avec audit complet de tous les ordres

Inconvénients

Complexité opérationnelle 5 services + Redis + 5 bases PostgreSQL

Latence réseau Appels HTTP et latence événementielle

Cohérence éventuelle Les états sont cohérents mais pas instantanément

Architecture technique

Event-driven Redis pub/sub avec `brokerx-events` crate

Event sourcing Tous les événements d'ordres persistés

Idempotence Table `processed_events` dans Order Service

Observabilité Prometheus + Grafana pour métriques distribuées

12.1.6 Authentification JWT avec MFA (multi-factor authentication)

Statut Acceptée

Contexte Le système BrokerX gère des transactions financières sensibles nécessitant une authentification robuste. Nous devons identifier les utilisateurs de manière sécurisée tout en maintenant de bonnes performances sous forte charge.

Décision Authentification via JSON Web Tokens (JWT) obtenu après une vérification en deux étapes (MFA) utilisant le mot de passe et un code à usage unique (OTP) envoyé par email.

Processus d'authentification

1. Login initial

```
Client → API Gateway : POST /v1/auth/login (email, password)
         API Gateway → User Service : Vérification credentials
         API Gateway → MFA Adapter : Génération code OTP (6 chiffres)
         MFA Adapter → Email SMTP : Envoi code
         API Gateway → Client : challenge_id
```

2. Vérification MFA

```
Client → API Gateway : POST /v1/auth/verify-mfa (challenge_id, code)
         API Gateway → MFA Adapter : Validation code
         API Gateway → Client : JWT token
```

3. Requêtes authentifiées

```
Client → API Gateway : Header "Authorization: Bearer <JWT>"
         Middleware : Validation JWT + extraction user_id
         Handler : Accès aux ressources de l'utilisateur
```

Composants

JWT Service (`api_gateway/src/auth.rs`)

- Génération de tokens signés avec secret partagé
- Validation et décodage des tokens
- Expiration configurable (24h par défaut)
- Claims : `sub` (email), `exp` (expiration), `iat` (issued at)

Email Adapter (`notification_service/`)

- Génération de codes OTP à 6 chiffres
- Stockage temporaire en mémoire (HashMap)
- Expiration des challenges (5 minutes)
- Envoi via SMTP (Gmail par défaut)

Middleware d'authentification (`api_gateway/src/middleware.rs`)

- Extraction du token depuis header `Authorization`
- Validation du token
- Injection du `user_id` dans les extensions de la requête
- Retour 401 Unauthorized si invalide

Configuration

Variables d'environnement

`JWT_SECRET` Clé secrète pour signer les tokens

`JWT_EXPIRY_HOURS` Durée de validité (défaut: 24)

`MFA_BYPASS` Mode test sans vérification MFA (défaut: false)

`SMTP_*` Configuration email (serveur, credentials)

Mode MFA Bypass Pour les tests de charge, le mode `MFA_BYPASS=true` permet :

- Acceptation de n'importe quel code (ex: « 000000 »)
- Pas d'envoi d'email réel
- Retour immédiat du JWT

Conséquences

Avantages

Sécurité renforcée Double facteur d'authentification

Stateless JWT ne nécessite pas de stockage serveur

Performance Validation locale du JWT (pas d'appel DB)

Testabilité Mode bypass pour load testing

Inconvénients

Latence MFA Délai d'envoi email (1-3 secondes)

Stockage temporaire Challenges MFA en mémoire (perdus au redémarrage)

Révocation complexe Impossible d'invalider un JWT avant expiration en l'état

12.1.7 Transactional Outbox Pattern pour publication fiable d'événements

Statut Acceptée

Contexte

Dans une architecture événementielle, garantir que les événements sont publiés de manière fiable est critique. Le problème classique :

1. Service met à jour sa base de données
2. Service publie un événement vers Redis

3. **CRASH** entre les deux étapes

Conséquence La base de données est mise à jour mais l'événement n'est jamais publié, créant une incohérence permanente.

Décision

Implémentation du **Transactional Outbox Pattern** dans Event Store Service pour garantir l'atomicité entre mise à jour locale et publication d'événements.

Implémentation

Principe

1. Transaction unique : L'événement est sauvegardé dans `order_events` ET `outbox` dans la même transaction PostgreSQL
2. Worker asynchrone : Un worker poll la table `outbox` toutes les 100ms et publie vers Redis
3. Idempotence : Chaque événement a un `event_id` unique pour éviter les duplications

Schema de la table Outbox

```
CREATE TABLE outbox (
    id BIGSERIAL PRIMARY KEY,
    order_id UUID NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    processed_at TIMESTAMPTZ,
    processing_attempts INTEGER DEFAULT 0,
    last_error TEXT
);
```

Flux de traitement

API Gateway -> Event Store Service -> Transaction BEGIN -> INSERT INTO order_events -> INSERT INTO outbox -> Transaction COMMIT -> [Outbox Worker] -> PUBLISH to Redis -> Mark processed

Conséquences

Avantages

Fiabilité Publication garantie même en cas de crash

Cohérence Événements et base de données toujours synchronisés

Résilience Retry automatique en cas d'erreur réseau

Observabilité Monitoring des événements en attente

Inconvénients

1. Latence : Délai de polling (100ms)
2. Complexité : Worker supplémentaire à maintenir
3. Stockage : Événements dupliqués dans `order_events` et `outbox`

Implémentation dans BrokerX

Localisation Event Store Service
Worker Tâche Tokio polling toutes les 100ms
Métriques Prometheus pour surveillance du backlog
Cleanup Événements publiés supprimés après 7 jours

12.1.8 Event Sourcing pour audit immutable et traçabilité complète

Statut Acceptée

Contexte Dans un système de trading, la traçabilité complète des ordres est essentielle pour :

1. Audit réglementaire : Historique immutable de toutes les transactions
2. Debugging : Comprendre exactement ce qui s'est passé
3. Reconstitution d'état : Capacité à recalculer l'état à tout moment

Le modèle CRUD traditionnel ne convient pas car les mises à jour écrasent les valeurs précédentes.

Décision

Implémentation d'Event Sourcing : stocker la séquence d'événements au lieu de l'état actuel.

Architecture

Table d'événements

```
CREATE TABLE order_events (
    sequence_number BIGSERIAL PRIMARY KEY,
    event_id UUID NOT NULL UNIQUE,
    order_id UUID NOT NULL,
    user_id UUID NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    symbol VARCHAR(10) NOT NULL,
    quantity BIGINT NOT NULL,
    order_side VARCHAR(10) NOT NULL,
    order_type JSONB NOT NULL,
    filled_quantity BIGINT,
    average_price DECIMAL(15, 2),
    occurred_at TIMESTAMPTZ NOT NULL,
    correlation_id UUID NOT NULL,
    causation_id UUID,
    metadata JSONB
);

-- Immutabilité garantie
CREATE RULE no_updates AS ON UPDATE TO order_events DO INSTEAD NOTHING;
CREATE RULE no_deletes AS ON DELETE TO order_events DO INSTEAD NOTHING;
```

Types d'événements Les événements métier capturent ce qui s'est passé :

`OrderCreated` Nouvel ordre créé
`OrderValidated` Validation pré-trade réussie
`OrderRejected` Validation échouée
`OrderExecuted` Ordre exécuté au marché
`OrderPartiallyFilled` Exécution partielle
`OrderCompleted` Saga terminée
`OrderCancelled` Ordre annulé

Implémentation dans BrokerX

Le système stocke tous les événements dans PostgreSQL avec garantie d'immutabilité (règles SQL bloquant UPDATE/DELETE).

Requêtes d'audit

- Historique complet d'un ordre via `correlation_id`

- Reconstitution d'état à une date donnée (time travel)
- Traçabilité complète des sagas

Conséquences

Avantages

Audit immutable Historique complet impossible à modifier

Time travel Reconstitution de l'état à n'importe quel moment

Debugging Traçabilité complète des bugs

Compliance Satisfaction des exigences réglementaires

Inconvénients

Stockage Volume de données plus important